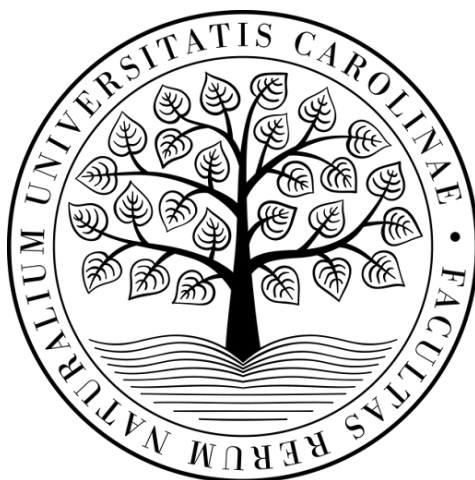


GEOINFORMATIKA EX.3

# Nekratší cesta grafem



KARLOVA UNIVERZITA  
Přírodovědecká fakulta

*Jan Kopečný*

Praha  
2025

## 1 Zadání:

Implementujte Dijkstra algoritmus pro nalezení nejkratší cesty mezi dvěma uzly grafu. Vstupní data budou představována silniční sítí doplněnou vybranými sídly. Otestujte různé varianty volby ohodnocení hran grafu tak, aby nalezená cesta měla:

- nejkratší euklidovskou vzdálenost
- nejmenší transportní čas.

Ve vybraném GIS konvertujte podkladová data do grafové reprezentace představované neorientovaným grafem. Pro druhou variantu optimální cesty navrhnete vhodnou metriku, která zohledňuje rozdílnou dobu jízdy na různých typech komunikací. Výsledky (dvě různé cesty pro každou variantu) umístěte do tabulky, vlastní cesty vizualizujte. Dosažené výsledky porovnejte s vybraným navigačním SW.

| Krok                                                 | Hodnocení  |
|------------------------------------------------------|------------|
| Dijkstra algoritmus.                                 | 20b        |
| Řešení úlohy pro grafy se záporným ohodnocením.      | +10b       |
| Nalezení nejkratších cest mezi všemi dvojicemi uzlů. | +10b       |
| Nalezení minimální kostry některou z metod.          | +15b       |
| Využití heuristiky Weighted Union                    | +5b        |
| Využití heuristiky Path Compression                  | +5b        |
| <b>Max celkem:</b>                                   | <b>65b</b> |

Table 1: Hodnocení kritérií kroků algoritmů

## 2 Zpracování

### 2.1 Vstupní data

Pro zadaný úkol byla využita podkladová data OpenStreetMap, dostupná v knihovně osmnx. Knihovna Osmnx umožňuje stažení dat ve formě Multi-Graph. Pro úkol byla vybrána síť dopravní komunikace v Karlovarském kraji. Zájmové území lze v osmnx definovat pomocí několika způsobů. V našem případě byl použit `osmnx.graph_from_place("Karlovarský kraj, Czechia", network_type="drive")`, který importuje silnice vymezené hranicemi kraje. Pro další práci byla tato data konvertována do slovníku se strukturou:

```
Graph={node1:{ neighbour1: {"distance": 8, "speedlimit": 50}}}
```

Pro komunikace byly zachovány atributy délky komunikace a cestovní rychlost, které budou sloužit pro výpočet nejmenšího transportního času. Tato konverze zajišťuje konzistenci klíčů a správnost datových typů, což je nezbytné pro efektivní výpočty nejkratších cest, minimální kostry či dalších analýz. Implementace dále předpokládá, že graf je neorientovaný – proto jsou duplicitní hrany eliminovány například pomocí podmínky, že hrana je přidána pouze v případě, kdy je uzel u menší než uzel v (u < v). Takto připravená data umožňují přesnou a efektivní analýzu reálné dopravní sítě.

Dále byl vytvořen slovník uzlů které reprezentují sídla. Sídla byla importována jako body osmnx, kterým byl přiřazen nejbližší uzel sítě komunikací za pomoci funkce `osmnx.distance.nearest_nodes(G,X,Y)`

## 2.2 Algoritmus Dijkstra

Dijkstrův algoritmus slouží k nalezení nejkratší cesty v grafu, pokud graf neobsahuje hrany se záporným hodnocením. Postupně prochází všechny uzly a pomocí tzv. relaxace hran aktualizuje nejkratší známou vzdálenost k ostatním uzlům. Algoritmus využívá prioritní frontu, do které ukládá uzly podle jejich aktuální vzdálenosti, a vždy vybere ten s nejmenší hodnotou pro další zpracování. Tento postup se opakuje, dokud nejsou všechny uzly zpracovány, čímž je zaručeno nalezení nejkratších cest.

## 2.3 Princip algoritmu Dijkstra

### 1. Inicializace

Každému uzlu je přiřazena počáteční hodnota vzdálenosti; výchozí uzlu se nastaví vzdálenost 0 a všem ostatním nekonečno.

Pro každý uzel v grafu se nastaví předchůdce (jako perent node nebo predecesor), který je pro počátek nastaven jako None

Prioritní fronta: Je iniciována prioritní fronta za pomoci min.heap, pro vybrání uzlu s nejmenší možnou vzdáleností.

### 2. Iterativní zpracování uzlů

Z prioritní fronty je opakovaně vybrán uzel s nejmenší vzdáleností (váhou). Pro každý sousední uzel vybraného uzlu se spočítá nová potenciální vzdálenost (relaxace)

### 3. Aktualizace hodnot

Pokud je nově vypočtená vzdálenost ke konkrétnímu sousedovi sníží dosavadní zaznamenanou vzdálenost tohoto uzlu, je vzdálenost aktualizována a aktuální uzel je uložen jako předchůdce. soused je vložen do prioritní fronty.

### 4. Ukončení a sestavení cesty

Proces se opakuje, dokud není prioritní struktura vyprázdněna nebo dokud nedosáhneme cílového uzlu. Po skončení algoritmu lze zpětným sledováním předchůdců rekonstruovat nejkratší cestu.

## 2.4 Výpočet vah pro Dijkstra

Algoritmus byl implementován s dvěma varianty výpočtu vah a to "distance" a "travel time". Metoda distance bere jako váhu jednoduše délku komunikace mezi dvěma uzly, zatímco "travel time" uvažuje výpočet na základě cestovní rychlosti pro danou komunikaci uvedenou OpenStreetMap a vzdálenosti jako:

$$\text{distance}(\text{weight}) = \frac{\text{distance (km)}}{\text{speed (km/h)}} \quad (1)$$

## 2.5 Nalezení nejkratší vzdálenosti mezi všemi dvojicemi uzlů

Jako uzly byly v tomto případě použity sídla v Karlovarském kraji, jejichž souřadnicím byli přiřazeny nejbližší uzly v grafu. Funkce využívá metodu *combinations* z knihovny *itertools* k vygenerování všech možných dvojic měst, pro které se následně pomocí Dijkstrova algoritmu (funkce *DJ*) spočítá nejkratší cesta mezi odpovídajícími uzly. Výsledná cesta a její celková vzdálenost (případně doba jízdy při volbě režimu „travel time“) jsou rekonstruovány funkcí *reconstruct\_path* a uloženy do slovníku, který se poté exportuje do JSON souboru jako *"all\_shortest\_paths.json"*. Funkce dále umožňuje nastavení limitu jako *limit=n* prvních *n* dvojic ze slovníku *cities* pro které budou vzdálenosti počítány. Limitní počet je vhodné nastavit vzhledem k vysokému počtu uzlů sídel v grafu. Výpočet vzdálenosti mezi všemi uzly v grafu tak může být velmi výpočetně náročné. Funkci lze v kodu spustit jako:

```
results = calculate_combinations(G_dict, limit=4, method='distance')
```

## 2.6 Řešení úlohy pro grafy se záporným hodnocením - Bellman-Ford

Algoritmus Dijkstra není vhodný pro grafy obsahující hrany se zápornými váhami, protože v takovém případě může vrátit chybná řešení nebo nemusí vůbec dojít k ukončení algoritmu, v případě že záporná váha hrany umožní nekonečné snižování zaznamenané vzdálenosti (**negativní cyklus**). Pro tyto případy je vhodné využít Bellman-Fordův algoritmus, který najde nejkratší cesty v grafech se záporným ohodnocením a také identifikuje **negativní cykly**.

## 2.7 Princip a implementace algoritmu Bellman-Ford

V kodu je Bellman-Ford implementován jako funkce *bellman\_ford* která podobně jako algoritmus *dijkstra* vrací slovník *distances* a *predecessors*, z kterých je následně možné rekonstruovat cestu grafem. Funkci lze zavolat jako:

```
distances, predecessors = bellman_ford(G_dict, start_node, method='distance')
```

Jednotlivé kroky jsou v kodu implementovány následovně:

### 1. Inicializace:

Pro každý uzel nastavíme počáteční hodnotu vzdálenosti *distances* na nekonečno, kromě počátečního uzlu, kde je vzdálenost 0. Současně si zaznamenáváme předchůdce *predecessors* každého uzlu, zpravidla jako *None*.

### 2. Opakovaná relaxace hran:

Algoritmus postupně projde všechny hrany grafu a pro každou hranu (*u, v*) aktualizuje vzdálenost do uzlu *v*, pokud nalezne kratší cestu skrze uzel *u*. Tento krok se opakuje celkem  $(n - 1)$  krát (kde  $(n)$  je počet uzlů *num\_nodes*), protože nejkratší cesta může obsahovat maximálně  $(n-1)$  hran.

Dále algoritmus spočítá váhu hrany a to metodou *"distance"* nebo *"travel time"*, které jsou popsány v kapitole 2.4.

Pokud součet aktuální vzdálenosti uzlu a váhy hrany zlepší (sníží) dosud známou vzdálenost k sousednímu uzlu, aktualizuje se hodnota v *distances* a uloží se aktuální uzel jako předchůdce v *predecessors*.

### 3. Kontrola záporného cyklu:

Po provedení konečného počtu  $(n-1)$  iterací se provede se znova projdou všechny hrany, pokud se najde hrana u které lze vzdálenost zkrátit, znamená to že graf obsahuje negativní cyklus. V tom případě algoritmus vyvolá výjimku a upozorní na tuto situaci.

```
ValueError("The graph contains a negative weight cycle")
```

### 4. Návrat výsledků:

okud není nalezen žádný záporný cyklus, funkce vrátí slovníky *distances* a *predecessors*

## 2.8 Nalezení minimální kostry grafu (Kruskalův algorytmus)

Kruskalův algoritmus je algoritmus používaný k nalezení nejmenší kostry grafu (Minimum spanning tree) U souvislého grafu hledá podmnožinu hran, která tvoří strom obsahující všechny uzly, s tím, že celková váha (součet délek) hran grafu je minimální.

### 1. Sestavení seznamu hran:

Nejdříve kod projde celý krav a vytvoří seznam hran `edges=[]`, kde je každá hrana reprezentována jako trojice (`weigh, node, neighbour`) tedy jako váha, uzel a soused se kterým je propojen. Váha hrany se vypočítá podle zvolené metody – buď přímo jako délka komunikace (metoda "*distance*"), nebo jako odhad cestovního času (metoda "*travel time*"). Podmínka `if node < neighbour`: zajistí, že každá hrana bude do seznamu přidána pouze jednou.

### 2. Seřazení hran:

Kruskalův algoritmus využívá strategii greedy algoritmů, tedy v každém kroku vybírá tu hranu, která má nejnižší váhu a zároveň nezpůsobí vznik cyklu. Tento přístup maximalizuje přínos každého kroku tím, že postupně doplňuje kostru o hrany s minimálními náklady, což vede k nalezení minimální kostry celého grafu.

Seznam `eges` je tedy v kodu seřazen vzestupně podle váhy hran

### 3. Inicializace disjunktních množin:

Pro správu komponent grafu se vytváří dvě slovníkové struktury – `parent`, kde je každý uzel nejprve sám sobě rodičem, a `rank`, který slouží k optimalizaci sloučení (`union`) pomocí metody `union-by-rank`.

### 4. Funkce `find` a `union`:

- funkce **`find(x)`** slouží ke zjištění reprezentanta (kořene) množiny, do které uzel patří, a zároveň využívá techniku zkracování cest (**`path compression`**) pro zvýšení efektivity.
- Funkce **`union(x, y)`** pak spojuje dvě množiny (komponenty) na základě jejich ranků. Pokud jsou oba uzly ve stejné množině, nedojde ke sloučení, což indikuje vznik cyklu.(využití heuristiky **`weighted union`**)

## 2.9 Použití heuristik **Weighted Union** a **Path Compression**:

**Weighted Union** (`union by rank`) zajišťuje, že při slučování dvou množin se vždy připojí menší strom (s nižším hodnocením) ke stromu s vyšším hodnocením, čímž se udržuje výsledek slučování co nejvíce vyvážený a strom má menší hloubku. **Path Compression** dále optimalizuje operaci hledání (`find`) tím, že během procházení cesty k jejímu kořeni aktualizuje rodiče všech navštívených uzlů tak, aby přímo ukazovaly na kořen. To výrazně zkracuje budoucí operace `find`. Obě techniky tak umožňují rychlý průběh kruskova algorytmu.

## 3 Popis výsledků

### 3.1 Dijkstra algoritmus

### 3.2 Trasa Cheb - Kynšperk nad Ohří

Algoritmus Dijkstra byl testován v obounastaveních a to na krátké trase Cheb - Kynšperk nad ohří, dály byli výsledky porovnány s trasy aplikace Mapy.cz. Rozdílné trasy nalezené při nastavení metody výpočtu vah jsou vidět ve Figure 1. kde je patrné že metoda "distance" správně volí eouklidovsly kradší cestu, zatímco metoda výpočtu "travel time" zohledňuje cestovní rychlost a volí cestu po rychlostní silnici.

V tabulce Table 2 je možné porovnat výsledky našeho algoritmu djikstra s výseledky které pro trasu uvádí aplikace Mapy.cz. Mapy.cz automaticky zohledňují cestovní čas, takže je vhodné ho porovnat s cestovním časem algoritmu Dijkstra, který uvádí podobný výsledek 12,37 minut. Nepatrný rozdíl muže být způsoben rozdílem lokací počátečních a koncových uzlů mezi mapy.cz a naším Dijkstra algorytmem. Dále je možné že Mapy.cz uvažují pro výpočet maximální rychlosti namísto průměrných cestovních rychlostí použitý pro výpočet v implementovaném Dijkstra.

|          | method      | distance | time      |
|----------|-------------|----------|-----------|
| Dijkstra | distance    | 15.78 km |           |
| Dijkstra | travel time |          | 12.37 min |
| Mapy.cz  |             | 15.8 km  | 13 min    |

Table 2: porovnání výsledků implementace djikstra a Mapy.cz Trasa Cheb - kynšperk

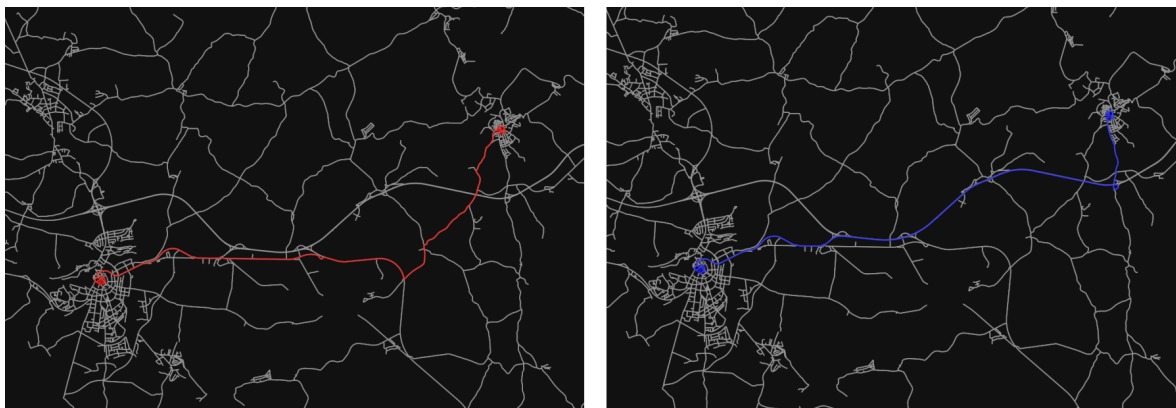


Figure 1: Cesta Cheb - Kynšperk nad Ohří při nastavení 'distance'(červeně) a 'travel time' modře

### 3.3 Trasa Cheb - Karlovy vary

Dále byl algoritmus testován na trase Cheb - Karlovy vary. Výsledky jsou k vidění v Table 3. Na delší trase se výpočet cestovní doby liší o něco víc než na kratší trase Cheb - Kynšperk nad O. Vzhledem k delší trase uvažovanou naším Dijkstra algorytmem a kratší cestovní době (cca. 3 min) je možné že rozdíl je způsoben tím že náš algoritmus neuvažuje "klikatost" komunikace a předpokládá tedy vyšší cestovní rychlost v porovnání s odhadem Mapy.cz. Další možností je, že Mapy.cz uvažují situace jako vytíženost dopravy, či omezení na komunikaci, které nejsou pro výpočet Dijkstra zahrnuty. Na)lezené trasy podle metod "distance" a "travel time" jsou v tomto případě totožné, jak je patrné z Figure 2.

|          | method      | distance | time      |
|----------|-------------|----------|-----------|
| Dijkstra | distance    | 44.14 km |           |
| Dijkstra | travel time |          | 25.77 min |
| Mapy.cz  |             | 43.1 km  | 29 min    |

Table 3: porovnání výsledků implementace djikstra a Mapy.cz. Trasa Cheb - Karlovy vary

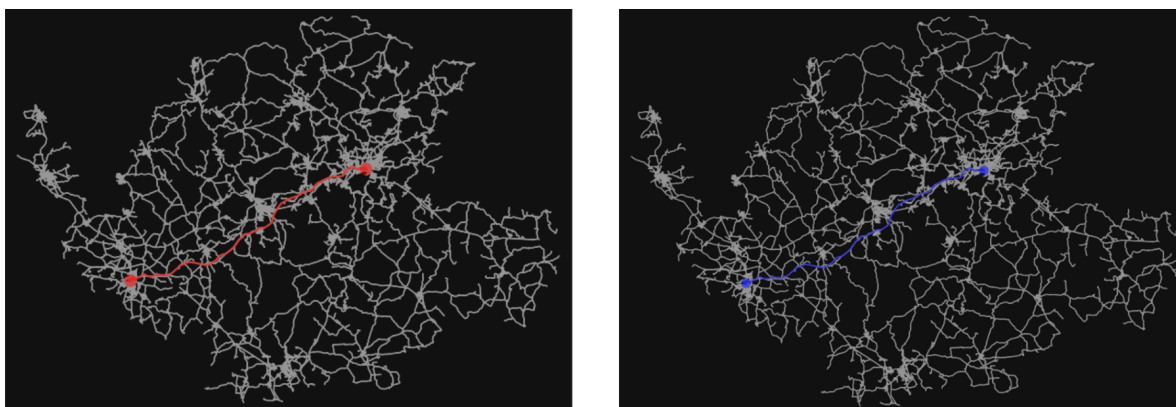


Figure 2: Cesta Cheb - Kynšperk nad Ohří při nastavení 'distance'(červeně) a 'travel time' modře

## 4 Zdroje:

W3Schools. (2025). *Kruskal's Minimum Spanning Tree Algorithm*. Published by W3Schools, 2025. Available at: [https://www.w3schools.com/dsa/dsa\\_algo\\_mst\\_kruskal.php](https://www.w3schools.com/dsa/dsa_algo_mst_kruskal.php).

Algoritmy.net (2024c). *Kruskaluv algoritmus*. Dostupn´e online, navštíveno 6. února 2025. url: <https://www.algoritmy.net/article/1417/Kruskaluv-algoritmus>.

Algoritmy.net (2024b). *Dijkstruv algoritmus*. Dostupn´e online, navštíveno 1. února 2025. url: <https://www.algoritmy.net/article/5108/Dijkstruv-algoritmus>.